Systèmes d'exploitation (Charleroi)
Projet — Tri parallèle

Pierre Vandenhove (pierre.vandenhove@umons.ac.be)
Gauvain Devillez (gauvain.devillez@umons.ac.be)

Année académique 2025–2026

Début : 7 avril 2026
Remise : **mercredi 13 mai 2026 à 23h59**
Le but de ce projet est de coder une application permettant le tri efficace, **parallélisé**, d'entrées d'un fichier. Les objectifs sont les suivants :

- se familiariser avec du code et des structures de données concurrentes ;
- utiliser l'interface POSIX pour manipuler des threads et des primitives de synchronisation ;
- utiliser l'interface POSIX pour manipuler des fichiers.

1 Spécification

On suppose un format de fichiers contenant un certain nombre d'*entrées*. Chaque entrée fait une taille fixe d'exactement 100 bytes. Les quatre premiers bytes de chaque entrée représentent une *clé* que nous allons utiliser pour trier les entrées. Les clés sont simplement ordonnées par ordre lexicographique des quatre bytes, lus de gauche à droite. Chaque entrée est donc composée de deux parties : 4 bytes pour la clé et 96 bytes pour le reste de l'entrée (la *valeur*). Ce genre de formats assez rigide est parfois utilisé pour des applications dont la performance est critique, comme par exemple pour des bases de données ou dans des systèmes plus anciens. [1] Un avantage est que cela simplifie l'accès aux différentes entrées du fichier, car on peut facilement calculer la position de chaque entrée, juste en connaissant la taille des entrées, et sans devoir parcourir tout le fichier. Par exemple, si l'on veut accéder au byte 64 de l'entrée 37, il suffit d'accéder au byte $37 \times 100 + 64 = 3764$ du fichier.

Vous implémenterez un programme en C dans un fichier `psort.c` qui trie dans l'ordre croissant les entrées d'un fichier (respectant le format ci-dessus) selon leurs clés. Le résultat sera écrit dans un fichier de sortie. Votre programme prendra trois arguments via la ligne de commande :

```
$ ./psort input output nb_threads
```

Un tri réussi va lire toutes les entrées du fichier `input`, les trier selon les clés, et écrire le résultat dans le fichier `output`. Le fichier `input` **ne** doit **pas** être modifié par le programme. Le nombre maximal de threads (≥ 1) à utiliser en parallèle dans votre programme est donné par l'argument `nb_threads`. Utilisez la fonction `fsync()` pour forcer l'écriture de votre fichier `output` sur le disque avant de terminer le programme.

2 Considérations de base

Vous pouvez supposer que les données de chaque fichier tiennent complètement dans la mémoire de l'ordinateur (et tiennent même *deux fois* dans la mémoire), ce qui permet un tri en une seule passe. [2]

Les contraintes minimales pour la réussite du projet sont les suivantes.

- L'algorithme de tri parallèle doit être codé **en C par vous-même(s)**, sans utiliser de fonction de tri provenant d'une bibliothèque tierce (**ne pas utiliser la fonction** `qsort()` **!**). Vous pouvez bien sûr utiliser d'autres fonctions des librairies standard de C, comme l'interface POSIX pour les threads et les fichiers.
- Le tri doit exploiter avantageusement le parallélisme en utilisant des threads POSIX. Cela signifie que le temps d'exécution de votre programme sur de grands fichiers doit s'améliorer quand on augmente (modérément) le nombre de threads utilisés. Vous pouvez supposer que le tri sera effectué sur un ordinateur moderne avec au moins 8 CPUs. Vous pouvez utiliser la fonction `time` pour mesurer le temps d'exécution

1. Comparez par exemple au format utilisé pour stocker les *inodes* dans le système de fichiers `vsfs` du Chapitre 4.
2. Lorsque toutes les données ne tiennent pas dans la mémoire vive, il faut utiliser un algorithme de tri parfois dit *externe* ou *multi-pass*. Dans ce cas, on divise les données en plusieurs parties, on trie chaque partie séparément, puis on fusionne les parties triées sans jamais utiliser trop de mémoire. Vous **ne** devez **pas** implémenter ce type de tri pour ce projet.

de votre programme ; c'est le temps `real` qui nous intéresse ici. Il est normal que le temps `user` augmente avec le nombre de threads, car la parallélisation a un surcoût qui implique que davantage d'instructions sont exécutées ; par contre, le temps réel devrait diminuer.

— L'algorithme doit pouvoir trier un fichier de 1 GB (donc avec environ 10 millions d'entrées) en un temps *raisonnable*, c'est-à-dire en moins d'une minute sur un ordinateur moderne. Cela **ne** vous permettra **pas** d'utiliser un algorithme de tri de complexité moyenne quadratique ($O(n^2)$) comme le tri par insertion ou le tri à bulles. Certains algorithmes vus dans des cours précédents peuvent être adaptés ; choisissez un algorithme qui peut être parallélisé. Bien sûr, au plus rapide, au mieux ! Un temps beaucoup plus court qu'une minute est possible. [3]

3 Considérations avancées

Pour aller plus loin, voici quelques suggestions de fonctionnalités supplémentaires.

— L'allocation du travail aux différents threads peut être faite de plusieurs façons. Celle-ci peut être faite de façon "statique", c'est-à-dire en divisant le travail entre les threads de façon fixée au début du programme, ou de façon "dynamique", c'est-à-dire en allouant des tâches aux threads au fur et à mesure que ceux-ci sont disponibles. Il est souvent plus avantageux de faire une allocation dynamique des tâches : cela permet de donner plus de travail aux threads qui sont en pratique plus rapides. Par contre, cela nécessite une gestion de la concurrence plus complexe pour synchroniser les threads.

— Il existe de nombreuses ressources sur l'optimisation d'algorithmes de tri dans des architectures parallèles. Indiquez en commentaire dans l'en-tête de votre fichier toutes les ressources utilisées pour concevoir votre algorithme (y compris les assistants IA), et pour chacune, ce qu'elle vous a apporté.

— Sur les systèmes Unix, il y a plusieurs façons de lire et d'écrire des fichiers. Des exemples sont les fonctions `fopen`, `fread`, `fwrite`, `fclose` et `fseek`. Pour notre problème, une autre façon efficace de lire et d'écrire des fichiers est d'utiliser des fichiers *memory-mapped*, via `mmap()`. L'idée est de "mapper" le fichier directement en mémoire, afin de pouvoir manipuler des bytes du fichier comme s'ils étaient dans un tableau en mémoire. Cela n'a pas été vu en cours, mais vous pourrez trouver de la documentation sur Internet et via la commande `man mmap`.

4 Tests

Nous vous fournissons un script PYTHON3 `generate_benchmark.py` permettant de générer des fichiers de test respectant le format ci-dessus pour votre programme. Vous pouvez passer en paramètre le nombre d'entrées à générer, et le script va créer un fichier `benchmark_n.txt` contenant n entrées aléatoires. Par exemple, pour générer un fichier de $\sim$ 500 MB, vous pouvez exécuter la commande suivante :

```
$ python3 generate_benchmark.py 5000000
```

Pour faciliter la visualisation du fichier et du tri, nous terminons chaque entrée par un caractère `\n` (qui est le centième byte de chaque entrée). Toutefois, **ne** vous basez **pas** sur cette information pour distinguer les entrées ; basez-vous uniquement sur le fait que chaque entrée fait exactement 100 bytes, comme spécifié plus haut, et ne supposez pas que le centième byte est nécessairement un caractère `\n`.

5 Modalités et remise

Le projet est à remettre sur Moodle pour le **mercredi 13 mai 2026 à 23h59** au plus tard. Vous devez remettre le fichier `psort.c` sur Moodle. Votre code doit être compilable avec les *flags* suivants : `-Wall -Werror -pthread -O2`. Dans le cas où vous utilisez plusieurs fichiers, vous devez remettre une archive au format `.zip` et expliquer l'organisation de votre code dans l'en-tête du fichier `psort.c`. Seul le code de votre projet doit être remis.

3. Comme point de comparaison, l'article classique implémentant AlphaSort, datant de 1994, triait un fichier d'1 GB en une minute. Nous n'attendons évidemment pas un algorithme aussi optimisé que celui de l'article, mais vous avez à votre disposition un ordinateur bien plus performant que celui utilisé en 1994 !